



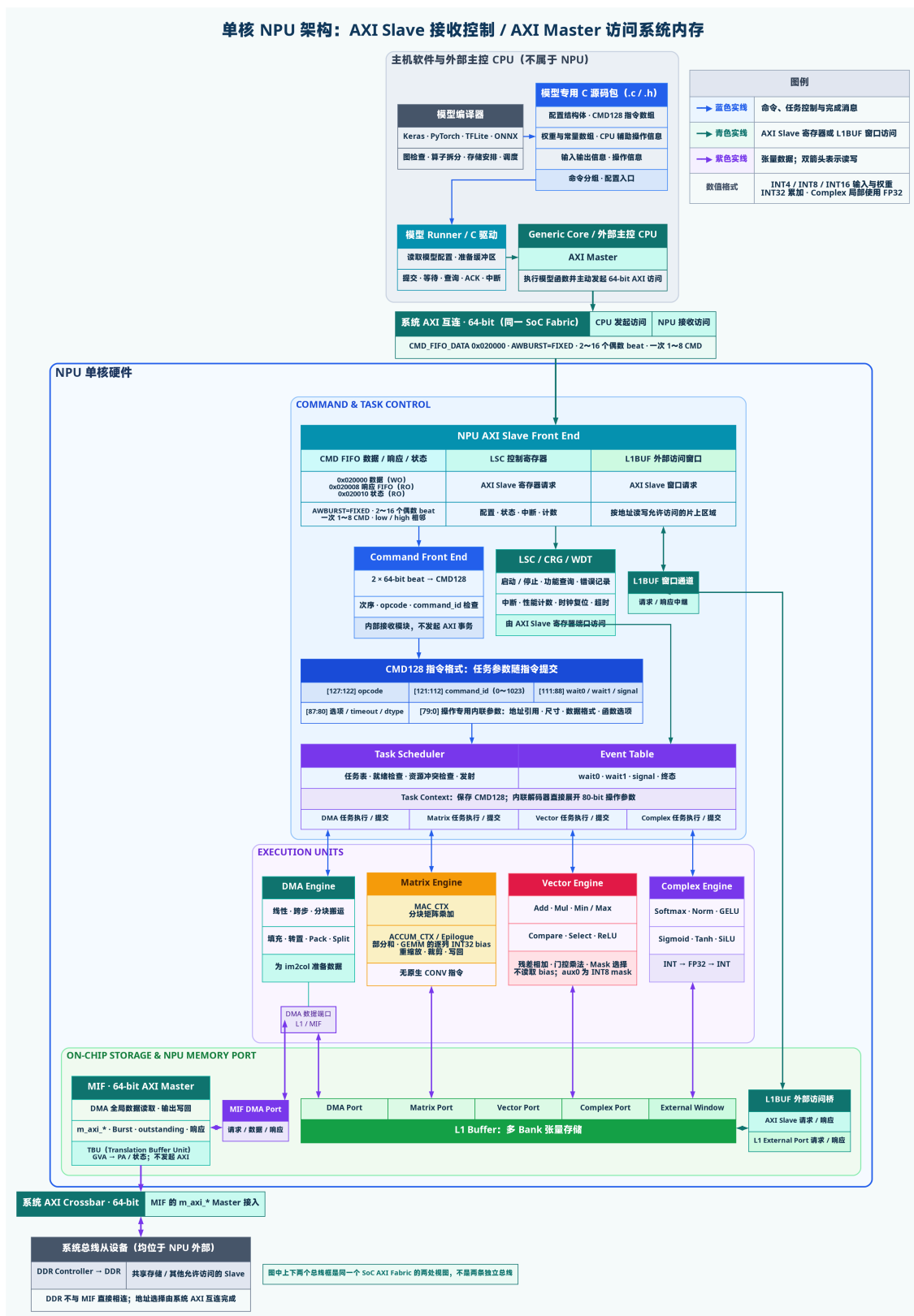
NPU 单核架构设计介绍

[!summary] 本设计从单核开始。外部 Generic Core 负责模型调度，NPU 通过 64-bit AXI Slave 接收控制访问和 CMD128，通过 64-bit AXI Master 访问系统内存。CMD128 把任务参数直接放入指令，不再从外部内存读取 Descriptor。

1. 设计目标

- 面向 Transformer、RNN、GRU、LSTM 和常见全连接网络。
- 软件可见张量支持 INT4、INT8、INT16、INT32；Matrix 使用 INT32 保存乘加结果。
- Sigmoid、Tanh、GELU、SiLU、Softmax 和 Norm 在 Complex Engine 内部执行 $INT \rightarrow FP32 \rightarrow INT$ ，FP32 中间值不写入软件可见张量。
- 总线数据宽度为 64 bit；一条 128-bit 指令使用相邻的 low、high 两个 beat。
- 本设计不设置卷积执行单元。Conv2D 由编译器拆成数据整理、im2col、GEMM、bias 和输出整理；CPU 可处理硬件指令不适合承担的步骤。
- Matrix 的分块乘加与 INT32 部分和处理可同时推进，减少执行单元等待。

2. 单核总体结构



图中的 Generic Core 是 NPU 外部的可控 CPU。CPU 运行模型函数和 C 驱动, 以 AXI Master 身份访问 NPU 的 AXI Slave。NPU 内的 MIF 以 AXI Master 身份接入 同一个 SoC AXI Fabric。图中上下两个总线框只是同一系统互连的两处视图。

[note] DDR Controller 位于 NPU 外部, 也接在系统 AXI Fabric 上。MIF 发出的地址由 系统互连送往 DDR Controller、共享存储或其他允许访问的设备。

3. 外部 CPU 如何提交指令

NPU AXI Slave 提供三类访问入口：

入口	访问方式	用途
CMD_FIFO_DATA	固定地址、AWBURST=FIXED	提交 CMD128
LSC 控制寄存器	普通 AXI 读写	启动、停止、状态、中断、基地址和计数器
L1BUF 外部窗口	普通 AXI 读写	调试或软件准备少量片上数据

驱动把每条 CMD128 排成 low word、high word。一次 FIXED burst 包含 2~16 个 偶数 beat，可提交 1~8 条完整指令。NPU 先检查 burst 长度、beat 次序和写响应， 再把完整指令交给 Command Front End。

[!warning] low word 与 high word 不得被另一条指令插入。外部 CPU 不直接连接 Task Scheduler，也不能绕过 Command Front End 写任务表。

4. 指令集设计

4.1 公共头部

位段	字段	说明
[127:122]	opcode	6-bit 操作码
[121:112]	command_id	10-bit 在途任务编号，范围 0~1023
[111:104]	wait_event0	第一个前置 Event ID，0xFF 表示不用
[103:96]	wait_event1	第二个前置 Event ID，0xFF 表示不用
[95:88]	signal_event	完成后更新的 Event ID，0xFF 表示不用
[87]	irq_success	任务成功时请求中断
[86]	irq_error	任务失败时请求中断
[85]	strict_numeric	复杂函数出现非法数值时返回错误
[84]	ordered	要求按提交次序检查发射条件
[83:82]	timeout_class	选择四档超时配置
[81:80]	dtype	0=INT4、1=INT8、2=INT32、3=INT16
[79:0]	payload	按操作类型解释的任务参数

opcode 占用最高 6 bit。数值 0~31 的含义见下表；32~63 返回 ILLEGAL_OPCODE。

CFE 和 TS 保存收到的 16 字节 CMD128，内联解码器从 payload 直接得到 地址引用、尺寸、数据格式和函数选项。opcode[127:122] 是线上 6-bit 紧凑值，不是执行单元内部枚举；全部任务操作数都在命令头和 payload 内。

4.2 操作码

当前定义的 32 个操作码如下。除标注为功能位关闭的两项外，其他表项均为 P0：

值	指令	执行单元	主要作用	关键输入、参数与输出	当前支持情况
0	NOP	Control	创建一个不进行数据运算的任务，可用于占位或验证任务控制。	payload=0，三个 Event 字段均为 0xFF；完成后只产生任务状态。	已实现
1	EVENT_SIGNAL	Control	由指令主动把一个 Event 置为成功。	signal_event 必须有效，payload=0；不启动计算单元。	已实现
2	EVENT_REARM	Control	复用已经完成且没有等待者的 Event，并开始新的代次。	signal_event 指定待复用的 Event；Event 仍被占用时返回错误。	已实现
3	EVENT_JOIN	Control	把两个前置 Event 的结果合并后写入一个新 Event。	wait_event0、wait_event1、signal_event 均有效；join_mode=0 要求两者成功，join_mode=1 表示任一成功即可。	已实现
4	GLOBAL_FENCE	Control	等待所选执行单元中更早提交的任务全部结束。	engine_mask[3:0] 依次选择 DMA、Matrix、Vector、Complex，至少选择一项。	已实现
5	DMA_COPY_ID	DMA	DMA 连续复制一段元素，并可在不同整数宽度之间转换。	源/目标 AREF28、元素数、源/目标 dtype、INT4 起始半字节；变窄时执行饱和。	已实现
6	DMA_COPY_ND	DMA	复制连续保存的张量区域。	字段与 DMA_COPY_ID 相同；不含 rank、shape、src/dst stride，带间隔访问由编译器拆成多条任务。	已实现
7	DMA_FILL	DMA	用同一个数值连续填充目标区域。	目标 AREF28、元素数、fill_value[31:0] 和目标 dtype；不含多维 shape。	已实现
8	DMA_TRANSPOSE	DMA	将连续行优先二维数组从 [rows, columns] 改排为 [columns, rows]。	源/目标 AREF28、行数、列数、dtype 和 INT4 半字节选择；不含行间隔字段。	已实现
9	DMA_PACK	DMA	从等间隔数据段读取并连续写入目标区域。	源/目标 AREF28、段数、每段字节数、段间隔；后三项均为 8 bit，且段间隔不得小于段字节数。	已实现
10	DMA_SPLIT	DMA	从连续区域读取，并按固定间隔写入多个数据段。	源/目标 AREF28、段数、每段字节数、段间隔；后三项均为 8 bit。	已实现
11	GEMM	Matrix	计算矩阵乘法，可在写回前加 INT32 bias。	A、B、C 的 LREF14，bias 的 LREF12，M/N/K、B 数据格式、C dtype 和右移位数；无 residual、ReLU、逐输出通道重缩放字段。	已实现
12	BMM	Matrix	对连续保存的多组矩阵执行批量矩阵乘法。	A、B、C 的 LREF14，batch 数、M/N/K、B 数据格式、C dtype 和右移位数；不使用 bias。	已实现
13	GEMM_ACCUM	Matrix	计算矩阵乘法，并把新结果加到原有 C。	A、B、C、M/N/K；C 必须为 INT32，bias 和右移位数必须为 0。	已实现
14	GEMM_ZERO	Matrix	清零 C 指向的 INT32 矩阵区域，为后续部分和累加做准备。	C 的 LREF14 和 M/N；A、B、bias、右移位数均为 0。	已实现
15	VADD	Vector	两个张量逐元素相加。	src0、src1、dst、rows/length 和广播模式；输出保持公共输入 dtype。	已实现
16	VSUB	Vector	两个张量逐元素相减。	src0、src1、dst、rows/length 和广播模式。	已实现
17	VMUL	Vector	两个张量逐元素相乘。	src0、src1、dst、rows/length 和广播模式；结果保存为 INT32。	已实现
18	VFMA	Vector	计算 $src0 \times src1 + src2$ 。	三个输入、目标地址、rows/length 和三个广播模式；乘加结果为 INT32。	已实现
19	VMAX	Vector	逐元素选择两个输入中的较大值。	src0、src1、dst、rows/length 和广播模式。	已实现
20	VMIN	Vector	逐元素选择两个输入中的较小值。	src0、src1、dst、rows/length 和广播模式。	已实现
21	VCMP_I	Vector	按指定关系比较两个输入，并生成 INT8 mask。	src0/src1 按公共 dtype 比较；src2[15:13] 依次为 EQ、NE、LT、LE、GT、GE，低 13 bit 为 0；真写 1，假写 0。	已实现
22	SELECT	Vector	根据 INT8 mask 在 src0 与 src1 之间逐元素选择。	src2 指向 VCMP_I 的 mask；mask 非零选择 src0，mask 为 0 选择 src1。	已实现
23	CLAMP	Vector	把每个输入值限制到给定的最小值和最大值之间。	src1、src2 字段直接保存 signed16 下限和上限，不作为地址使用。	已实现
24	RELU	Vector	计算 $\max(src0, 0)$ 。	src0、dst 和 rows/length；未使用的输入字段必须为 0。	已实现
25	ACT	Complex	计算 Sigmoid、Tanh、GELU 或 SiLU。	函数编号、输入/输出 scale 指数、目标 dtype 和输入限制区间；内部使用 FP32。	已实现
26	SOFTMAX	Complex	对每一行计算 Softmax，并支持布尔 mask 或有效长度。	src0、可选 aux、dst、rows/length、mask 模式、scale 指数和目标 dtype；causal 模式当前返回命令字段错误。	已实现
27	NORM	Complex	计算 LayerNorm 或 RMSNorm。	输入、gamma、可选 beta、目标地址、rows/length、epsilon 档位、scale 指数和目标 dtype。	已实现
28	ROPE	Complex	为注意力中的 Q/K 数据执行旋转位置编码。	操作码和命令位置已经分配；功能寄存器未声明支持时不得启动 Complex Engine。	P1，功能位关闭，返回 ILLEGAL_OPCODE
29	STAT	Complex	对一行计算 SUM、MAX 或 SUMSQ，并写出 INT32 结果。	src0、dst、rows/length 和统计模式；每行输出一个值。	已实现
30	RECIP	Complex	计算倒数，供除法类公式使用。	操作码和命令位置已经分配；功能寄存器未声明支持时不得写目标数据。	P1，功能位关闭，返回 ILLEGAL_OPCODE
31	VADD_RESCALE	Complex	按各自 scale 还原两个输入，相加后按目标 scale 写回整数。	src0、aux、dst、rows/length、三个 scale 指数和目标 dtype。	已实现

数值 32~63 没有定义，CFE 必须返回 ILLEGAL_OPCODE。VROPE 和 VRECIP 只有在功能寄存器声明支持后才能执行，其余表项属于当前硬件功能。

4.2.1 与旧版指令表的差异

旧版正式表有 33 项，当前 CMD128 表有 32 项。唯一未进入当前可提交集合的旧版 指令是 P1 DMA_GATHER_ND；旧表的完整操作码 0x28 不能写入当前 CMD128，因为当前值 28 已是 VROPE。

旧版项目	当前处理
DMA_GATHER_ND	当前未分配可发射值，32~63 都返回 <code>ILLEGAL_OPCODE</code> 。后续如需加入，索引表引用、目标地址和传输参数必须直接写入 <code>CMD128</code> 。
带 stride 的 DMA_COPY_ND	当前只接受连续元素区；编译器拆为多条复制，或采用 <code>PACK</code> 、 <code>SPLIT</code> 、 <code>TRANSPOSE</code> 。
多维 FILL、带行间隔 TRANSPOSE、宽字段 PACK/SPLIT	当前采用连续访问，且 <code>PACK/SPLIT</code> 的三个长度字段均为 8 bit；超出时拆分任务。
GEMM residual、ReLU、逐输出通道重缩放	当前由 <code>GEMM_ACCUM</code> 、Vector/Complex 后处理或软件完成。
causal Softmax	<code>mask_mode=2</code> 当前返回命令字段错误；编译器使用 <code>boolean mask</code> 、 <code>valid length</code> 或拆分任务。

4.3 地址引用

DMA 使用 28-bit `AREF`：

```
[27]    space: 0=L1BUF, 1=全局地址
[26:24] base_select
[23:0]  byte_offset
```

L1BUF 使用 `base_select=0`，地址就是 24-bit 字节偏移。全局地址按 `base_register[base_select] + byte_offset` 计算。基地址编号为 `0=0`、`1=input`、`2=weight`、`3=work`、`4=output`、`5=kv`。
Matrix 使用 14-bit `LREF14`，实际 L1BUF 字节地址为 `LREF14 × 64`；Vector 和 Complex 使用 16-bit `LREF16`，实际地址为 `LREF16 × 16`。Matrix bias 使用 12-bit `LREF12`，实际地址为 `LREF12 × 64`，因此 bias 参数放在 L1BUF 低 256 KiB 内。引用值 0 在 Matrix bias 字段中表示“不使用 bias”。

4.4 DMA payload

指令	[29:0]
COPY_1D / COPY_ND	src AREF28、dst AREF28、count[19:0]、目的 dtype、INT4 起始半字节
FILL	dst AREF28、count[19:0]、fill_value[31:0]
TRANSPOSE_2D	src AREF28、dst AREF28、rows[7:0]、cols[7:0]、目的 dtype
PACK / SPLIT	src AREF28、dst AREF28、段数、每段字节数、段间隔，各 8 bit

内联 `COPY_ND` 表示连续多维数组。带跨步的 `COPY_ND` 由编译器展开成多条 `COPY_1D`，或在适用时改用 `PACK`、`SPLIT`。这样保留多维复制能力，又不需要外部参数块。

4.5 Matrix payload

GEMM、GEMM_ACCUM 和 GEMM_ZERO：

位段	字段
[79:66]、[65:52]、[51:38]	A、B、C 的 <code>LREF14</code>
[37:26]	bias 的 <code>LREF12</code> ，0 表示不用
[25:20]、[19:14]、[13:8]	M-1、N-1、K-1
[7]	<code>b_int4</code>
[6:5]	C 的 dtype
[4:0]	直接给出 0~31 的右移位数

当 A 为 INT8 且 `b_int4=1` 时，B 为 INT4；否则 B 与 A 使用相同格式。允许的乘法组合是 INT4×INT4、INT8×INT8、INT8×INT4 和 INT16×INT16。
BMM 使用 A、B、C 三个 `LREF14`，随后是 6-bit `batch-1`、`M-1`、`N-1`、`K-1`、1-bit `b_int4`、2-bit C dtype 和 5-bit 右移位数，最低 6 bit 写 0。

[example] 设 A 的形状是 [M,K]，B 的形状是 [K,N]，bias 的形状必须是 [N] 且元素为 INT32。对输出元素 $C_{i,j}$ ，硬件计算：

$$C_{i,j} = \text{cast}\left(\text{round}\left(\frac{\sum_{k=0}^{K-1} A_{i,k} B_{k,j} + b_j}{2^s}\right)\right)$$

同一个 b_j 会加到输出矩阵第 j 列的每一行，即 $C_{0,j}, C_{1,j}, \dots, C_{M-1,j}$ 。这里不是把一个 bias 数值加到全部输出元素，而是每个输出列使用自己对应的 bias。

4.6 Vector 与 Complex payload

Vector 公共格式为四个 `LREF16`：`src0`、`src1`、`src2`、`dst`，再保存 `rows-1`、`length-1` 和三个 2-bit 广播方式。CMP 把 `src2` 字段的高 3 bit 解释为 EQ、NE、LT、LE、GT、GE 选择，输出为 INT8 0/1 mask；SELECT 把 `src2` 解释为 INT8 mask，0 选择 `src0`，非 0 选择 `src1`。Vector 不读取 Matrix bias 字段。
Complex 公共格式为 `src0` `LREF16`、`aux` `LREF16`、`dst` `LREF16`、`rows-1`、`length-1` 和 19-bit 函数参数。函数参数保存激活类型、指数形式的 scale、目的 dtype、Softmax mask 类型、Norm 模式和 epsilon 档位。LayerNorm 的 `aux` 指向 gamma，beta 位于对齐后的 gamma 数据之后；gamma、beta 的形状均为 `[length]`，供全部 `rows` 行使用。

5. Event 是什么

Event 是硬件任务之间的完成状态记录，不是数据缓冲区，也不是中断编号。每个 Event Table 表项保存状态、当前代次、生产者和等待者数量。CMD128 只写 8-bit Event ID；TS 在接收指令时读取该表项的当前代次。

- `signal_event=x`: 任务被接收后保留 Event x；任务结束时写入成功或失败。
- `wait_event0/1=x`: 任务只有在 Event x 成功后才能发射。
- 前置 Event 失败：等待任务返回 `DEPENDENCY_FAILED`，不启动执行单元。
- `EVENT_JOIN`: 把两个前置 Event 的结果合成一个输出 Event。
- `EVENT_REARM`: 旧代次已结束且等待者数量为 0 时，使同一 ID 进入下一代次。
- `0xFF`: 该 Event 字段不用。

[!example] GEMM 的 `signal_event=7`，GELU 的 `wait_event0=7`。CPU 可以连续提交两条指令。GELU 会留在等待状态；GEMM 写完最后一项输出并报告成功后，TS 才把 GELU 发给 Complex Engine。CPU 不需要在两条指令之间轮询。

Event ID 可以复用。编译器只有在旧生产者及全部消费者完成后才插入 `EVENT_REARM`，并在生成的命令批次信息中要求主机先完成前一批任务，再提交含 `EVENT_REARM` 的批次。

6. 模块分工与功能时序

模块	主要功能
AXI Slave Front End	接收命令 burst、控制寄存器和 L1BUF 窗口访问
CFE	将两个 64-bit beat 组成 CMD128，检查格式和重复 <code>command_id</code>
Task Scheduler	保存 CMD128、检查 Event、选择可发射任务、记录终态
Task Context	每项保存 16 字节 CMD128，供内联解码与执行单元读取
DMA Engine	全局内存与 L1BUF 搬运、填充、转置、PACK、SPLIT
Matrix Engine	GEMM/BMM、INT32 部分和、bias 和整数写回
Vector Engine	整数逐元素运算、比较、mask 选择和广播
Complex Engine	激活、Softmax、Norm、统计和不同 scale 的加法
L1BUF	多 Bank 片上张量存储，连接四个执行单元和外部窗口
MIF / TBU	DMA 全局访问、地址转换服务、AXI burst 与响应处理
LSC / CRG / WDT	状态、中断、错误、时钟复位和超时控制

一条任务按以下次序推进：

1. AXI Slave 接收 low、high 两个 beat。
2. CFE 组成 CMD128，检查操作码和 `command_id`。
3. TS 分配任务项和 16 字节 Task Context，当拍不产生参数读取请求。
4. 内联解码器检查 payload、地址范围、尺寸和数据格式。
5. 前置 Event 成功且执行单元可接收时，TS 发射任务。
6. 执行单元从 L1BUF 取数；只有 DMA 需要访问全局内存时才使用 MIF。
7. 最后一项写响应完成后，执行单元提交终态。
8. TS 更新 Event、任务状态和中断；CPU 查询、等待并 ACK 任务。

7. 软件与硬件分工

模型编译器接收 Keras、PyTorch、TFLite 或 ONNX 模型，执行图检查、常量整理、算子拆分、L1BUF 分配、Event 安排和 CMD128 编码。部署结果是可参加 C/C++ 构建的模型专用 .c、.h：

```
模型专用 C 源码包
├── 配置结构体
├── CMD128 指令数组
├── 权重与常量数组
├── 输入输出和操作信息
├── 命令批次与主机等待要求
└── CPU 辅助操作信息
```

生成结果不含外部 Descriptor 数组，也不把裸二进制作为主要入口。C 驱动 负责寄存器访问、缓存维护、FIXED burst 提交、等待、查询、中断和 ACK。模型中 不适合由现有指令完成的步骤由生成的操作信息交给 CPU 运行；例如 Conv2D 的数据整理可能由软件和 DMA 共同完成，乘加部分交给 Matrix Engine。

8. 面向模型的算子组合

模型	主要硬件操作
Transformer	Q/K/V GEMM、BMM、Softmax、残差加法、LayerNorm、GELU
SimpleRNN	输入 GEMM、循环 GEMM、bias 加法、Tanh
GRU	六组 GEMM、Sigmoid、Tanh、逐元素乘法和状态组合
LSTM	八组 GEMM、四个门函数、逐元素乘法和状态组合
CNN	im2col、DMA 搬运、GEMM、bias、激活和输出整理